

Especificación de Requisitos de Software (SRS)

Proyecto: Uno (Versión Simplificada de Consola)

Versión: 1.2

Fecha: 26 de febrero de 2026

Elaborado por:

Felipe Franceschy López

González Ocampo Javier

Martínez Guerrero Juan Carlos

Martínez Vera María Izel

Orduña Luna Leonardo

1. Introducción

1.1. Propósito

Este documento describe de manera clara y estructurada los requisitos del software para la implementación del juego de cartas UNO en su versión simplificada de consola. Su objetivo es establecer un entendimiento común entre los miembros del equipo de desarrollo sobre las funcionalidades, el alcance y las restricciones del proyecto, sirviendo como guía durante todo el ciclo de vida del desarrollo.

1.2. Alcance del Producto

Se desarrollará una versión inicial y funcional del clásico juego de cartas **UNO**. Esta primera versión consistirá en un juego para un solo jugador que se ejecutará en entorno de consola (interfaz de línea de comandos), donde el jugador humano se enfrentará a la computadora.

Lo que SÍ incluye esta primera versión:

- Modo de juego: Jugador humano vs Computadora
- Cartas numéricas con valores del 0 al 9
- Cuatro colores: Rojo, Azul, Verde y Amarillo
- Reglas básicas de coincidencia: jugar una carta que coincida en color o en número con la carta en mesa
- Flujo completo del juego con turnos alternados
- Sistema de robo de cartas cuando no hay jugadas válidas
- Detección de fin de partida y anuncio del ganador

Lo que NO incluye esta primera versión:

- Cartas especiales (Reversa, Toma dos, Saltar, Comodín, etc.)
- Interfaz gráfica de usuario (GUI)
- Persistencia de datos (guardar/cargar partidas)
- Modo multijugador (más de un jugador humano)
- Funcionalidades de red o juego en línea

2. Descripción General

2.1. Perspectiva del Producto

El juego será una aplicación de consola autónoma desarrollada en Java, que no requerirá conexión a internet ni dependencias externas para su ejecución. Toda la interacción con el usuario se realizará a través de la línea de comandos, siguiendo un flujo de entrada/salida estándar.

2.2. Funcionalidad del Producto (Flujo del Software)

Inicio del juego:

- Se crea una baraja virtual que contiene cartas de cuatro colores (rojo, azul, verde y amarillo) con números del 0 al 9.
- Por cada color se incluye: una carta con el número 0 y dos cartas de cada número del 1 al 9, totalizando 76 cartas.
- La baraja se baraja aleatoriamente utilizando un algoritmo de mezcla.
- Se reparten 7 cartas al jugador humano y 7 cartas a la computadora.
- Se coloca una carta boca arriba en la mesa como carta inicial del juego.

Durante el turno del jugador:

- La pantalla muestra la carta actual en la mesa.
- Se identifica y muestra ÚNICAMENTE las cartas de la mano del jugador que sean jugables (mismo color o número que la carta en mesa), cada una con su índice correspondiente.
- El jugador puede seleccionar una carta válida escribiendo el número de índice.
- Si el jugador no tiene ninguna carta jugable, debe tomar cartas de la baraja UNA POR UNA. Cada carta robada se añade a su mano.
- Al obtener una carta jugable, se juega automáticamente y se convierte en la nueva carta en mesa, finalizando su turno.

Durante el turno de la computadora:

- La computadora busca en su mano la primera carta jugable.
- Si encuentra una carta válida, la juega y se muestra un mensaje indicando qué carta jugó.
- Si no tiene cartas válidas, roba cartas de la baraja UNA POR UNA, revisando después de cada robo si la carta obtenida es jugable.
- Este proceso continúa hasta encontrar una carta válida, la cual se juega automáticamente.

Fin del juego:

- El juego termina cuando un jugador (humano o computadora) se queda sin cartas.
- Se muestra un mensaje indicando al ganador: "!!! TÚ HAS GANADO !!!" o "!!! LA COMPUTADORA HAS GANADO !!!".

3. Requisitos Específicos

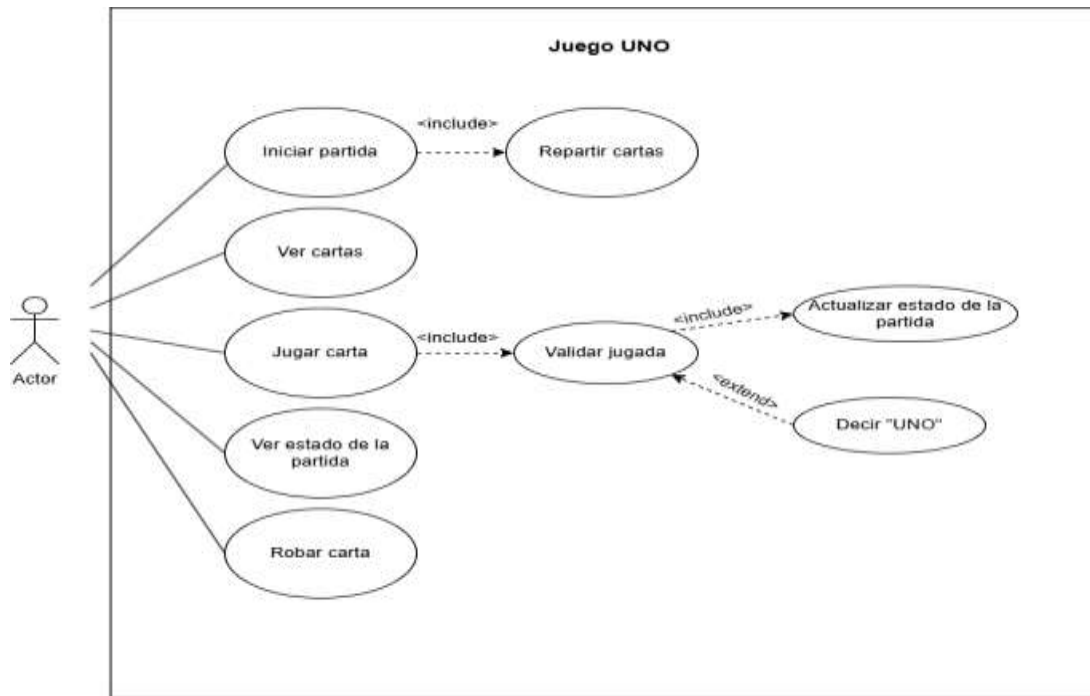
3.1. Requisitos Funcionales (RF)

ID	Requisito	Descripción
RF1	Creación de la Baraja	Crear una baraja de 76 cartas compuesta por 4 colores (rojo, azul, verde, amarillo). Por cada color incluir: 1 carta con número 0 y 2 cartas de cada número del 1 al 9. La baraja debe barajarse aleatoriamente al iniciar cada partida.
RF2	Reparto Inicial	Repartir 7 cartas al jugador y 7 cartas a la computadora, y colocar una carta boca arriba en la mesa como carta inicial.
RF3	Visualización en Consola	Mostrar en consola: la carta actual en mesa, las cartas válidas del jugador con su índice correspondiente (únicamente las jugables), mensajes de turno y las acciones realizadas por ambos jugadores.
RF4	Turno del Jugador	Permitir al jugador seleccionar una carta válida por su índice. Si no tiene cartas válidas, robar cartas de la baraja UNA POR UNA hasta obtener una carta jugable, la cual se juega automáticamente.
RF5	Turno de la Computadora	La computadora debe buscar y jugar la primera carta válida en su mano. Si no tiene cartas válidas, robar cartas de la baraja UNA POR UNA hasta encontrar una carta jugable, jugándola inmediatamente.
RF6	Validación de Jugadas	Validar que una carta sea jugable solo si coincide en color O en número con la carta en mesa.
RF7	Condición de Victoria	Detectar cuándo un jugador se queda sin cartas y mostrar el mensaje de victoria correspondiente.

3.2. Requisitos No Funcionales (RNF)

ID	Requisito	Descripción
RNF1	Lenguaje de Programación	Desarrollar en lenguaje Java, utilizando versión 8 o superior.
RNF2	Interfaz de Usuario	Interacción únicamente a través de consola (línea de comandos), sin uso de interfaces gráficas.
RNF3	Paradigma de Programación	Implementar el paradigma de Programación Orientada a Objetos (POO), aplicando principios como encapsulamiento, abstracción y modularidad.
RNF4	Calidad del Código	Código legible y mantenible aplicando buenas prácticas de programación: encapsulamiento (atributos privados con getters/setters), cohesión (clases con responsabilidades únicas) y bajo acoplamiento (dependencias mínimas entre clases). Incluir comentarios explicativos y nombres claros de variables y métodos.
RNF5	Control de Versiones	Gestionar el proyecto con Git para llevar un historial de cambios y facilitar la colaboración entre desarrolladores. Se recomienda usar plataformas como GitHub o GitLab.

4. CASOS DE USO DETALLADOS



CU-01: Iniciar Partida

Elemento	Descripción
Actor	Jugador Humano
Descripción	El jugador inicia una nueva partida ingresando su nombre
Precondición	El programa está ejecutándose
Flujo principal	1. El sistema muestra el título y reglas 2. El jugador ingresa su nombre 3. El sistema crea los jugadores y reparte cartas 4. El sistema muestra la carta inicial
Postcondición	La partida está lista para comenzar

CU-02: Ver Cartas

Elemento	Descripción
Actor	Jugador Humano
Descripción	El jugador visualiza las cartas en su mano
Precondición	El jugador tiene su turno activo
Flujo principal	1. El sistema muestra "TURNO DEL JUGADOR" 2. El sistema lista todas las cartas con su índice 3. El sistema muestra las cartas válidas para jugar
Postcondición	El jugador conoce sus opciones de juego

CU-03: Jugar Carta

Elemento	Descripción
Actor	Jugador Humano, Computadora
Descripción	El jugador selecciona una carta de su mano para jugarla
Precondición	El jugador tiene al menos una carta válida
Flujo principal	1. El jugador ingresa el índice de la carta 2. El sistema valida que la carta sea jugable 3. El sistema mueve la carta al mazo de descarte 4. El sistema actualiza el color y número actual
Flujos alternativos	Si la carta no es válida, el sistema muestra error y pide otra opción
Postcondición	La carta jugada está en el descarte y se actualizó el estado del juego

CU-04: Robar Carta

Elemento	Descripción
Actor	Jugador Humano, Computadora
Descripción	El jugador toma una carta del mazo principal
Precondición	El jugador no tiene cartas válidas O decide robar voluntariamente
Flujo principal	1. El jugador ingresa 't' para robar 2. El sistema toma una carta del mazo 3. El sistema muestra la carta robada 4. Si la carta es válida, se juega automáticamente 5. El jugador pierde su turno
Flujos alternativos	Si el mazo está vacío, se reconstruye desde el descarte
Postcondición	El jugador tiene una carta más en su mano y termina su turno

CU-05: Decir "UNO"

Elemento	Descripción
Actor	Jugador Humano
Descripción	El jugador declara "UNO" cuando le queda una sola carta
Precondición	El jugador tiene exactamente 1 carta en su mano
Flujo principal	1. El sistema detecta que el jugador tiene 1 carta 2. El sistema pregunta "¿Quieres decir UNO?" 3. El jugador responde 's' 4. El sistema registra que dijo UNO
Flujo alternativo	Si el jugador responde 'n', queda vulnerable a ser sorprendido
Postcondición	El jugador queda protegido o vulnerable según su respuesta

CU-06: Sorprender por no decir UNO

Elemento	Descripción
Actor	Jugador Humano, Computadora
Descripción	Un jugador sorprende a otro que no dijo UNO teniendo 1 carta
Precondición	El oponente tiene 1 carta y no dijo UNO
Flujo principal	1. El sistema detecta la infracción 2. Si el que sorprende es humano, pregunta "¿Quieres sorprender?" 3. El jugador acepta sorprender 4. El sistema aplica penalización de 2 cartas
Flujo alternativo	La computadora siempre sorprende automáticamente
Postcondición	El infractor roba 2 cartas y continúa el juego

CU-07: Recibir Penalización

Elemento	Descripción
Actor	Jugador Humano, Computadora
Descripción	Un jugador recibe cartas como castigo por no decir UNO
Precondición	El jugador fue sorprendido con 1 carta sin decir UNO
Flujo principal	1. El sistema anuncia la penalización 2. El jugador roba 2 cartas del mazo 3. El sistema muestra las cartas robadas
Postcondición	El jugador tiene 2 cartas adicionales en su mano

CU-08: Finalizar Partida

Elemento	Descripción
Actor	Jugador Humano, Computadora
Descripción	El juego termina cuando un jugador se queda sin cartas
Precondición	Un jugador juega su última carta
Flujo principal	1. El sistema detecta mano vacía 2. El sistema anuncia al ganador 3. El sistema muestra las manos finales 4. Pregunta si desea jugar otra vez
Flujo alternativo	Si el jugador responde 's', se reinicia la partida
Postcondición	El juego termina o se reinicia según la elección

6. MATRIZ DE TRAZABILIDAD CASOS DE USO - CLASES

Caso de Uso	Clases Involucradas	Métodos Principales
CU-01: Iniciar Partida	Main, JuegoUno	main(), JuegoUno()
CU-02: Ver Cartas	Jugador, Mano	mostrar(), obtenerCartasValidas()
CU-03: Jugar Carta	JuegoUno, Carta	turnoJugador(), jugarCarta()
CU-04: Robar Carta	JuegoUno, Mazo	tomarDelMazo(), tomarDelMazoConRetorno()
CU-05: Decir UNO	Jugador, JuegoUno	verificarReglaUno(), setDijoUno()
CU-06: Sorprender	JuegoUno	verificarReglaUno(), aplicarPenalizacionUno()
CU-07: Recibir Penalización	JuegoUno, Mazo	aplicarPenalizacionUno(), tomarDelMazo()
CU-08: Finalizar Partida	JuegoUno	jugar(), mostrarEstado()

ACTORES DEL SISTEMA

Actor	Descripción
Jugador Humano	Persona que interactúa con el juego a través de la consola. Puede ver cartas, jugar, robar, decir UNO y sorprender.
Computadora	Jugador automático controlado por el sistema. Juega siguiendo reglas predefinidas y siempre dice UNO automáticamente.
Sistema	El programa en sí, que gestiona las reglas, valida movimientos y mantiene el estado del juego.

6. Diagrama de Clases UML



7. Diccionario de Términos (Glosario)

Término	Definición	Representación en Código
Carta	Representa una carta individual del juego con color y número.	Clase Carta con atributos color (String) y numero (int).
Baraja (Mazo)	Conjunto de cartas disponibles para robar durante el juego.	Clase Mazo con atributo cartas (ArrayList<Carta>).
Mano	Conjunto de cartas que posee un jugador en un momento dado.	Clase Mano con atributo cartas (ArrayList<Carta>).
Jugador	Participante del juego, puede ser humano o computadora.	Clase Jugador con atributos nombre, mano, esHumano y dijoUno.
Juego	Controla el flujo principal, turnos y reglas del juego.	Clase JuegoUno que contiene la lógica completa del juego.

Término	Definición	Representación en Código
Principal	Punto de entrada del programa.	Clase Main con método main ().
Carta válida	Carta que puede jugarse porque coincide en color o número con la carta en mesa.	Método esValida(String colorMesa, int numeroMesa) en clase Carta.
Robar	Acción de tomar una carta de la baraja y agregarla a la mano.	Método tomarCarta() en clase Mazo y tomarCarta() en clase Jugador.
Jugar	Acción de quitar una carta de la mano y colocarla en la mesa.	Método jugarCarta(int indice) en clase Jugador.
Regla UNO	Regla que obliga a decir "UNO" cuando queda una sola carta.	Atributo dijoUno en clase Jugador y métodos verificarReglaUno() y aplicarPenalizacionUno() en clase JuegoUno.